

ESPECIFICACIÓN

Inconsistencias del Motor de Otorgamiento — política v1.1 ↔ implementación

Estado: auditoría cerrada · **los siete hallazgos resueltos e implementados** (INC-01 a INC-07, 11-09-2026). No queda ninguna decisión de negocio pendiente de este documento; si quedan los **parámetros abiertos heredados de la spec** (§5), que la propia política declara sin definir. **Fecha:** 10-09-2026 · **Alcance:** módulo de otorgamiento de NEX Factoring (Pipeline Comercial)

Re-verificado contra [main](#) **el 10-09-2026 (commit [212046f](#))**: los siete hallazgos se confirmaron uno por uno, y las referencias a línea de este documento están ancladas a ese commit. Los comandos de \$7 buscan por símbolo para que el documento no se vuelva a desanclar cuando el archivo crezca; sus **valores esperados** están actualizados al estado de hoy, con los previos al lado para poder comparar.

Corregidos e implementados el 11-09-2026. El diagnóstico de cada hallazgo se conserva íntegro —es lo que justifica la decisión— y arriba de cada uno va el recuadro con lo que el negocio decidió y lo que se implementó. Cada cierre tiene su caso en [tests_asignacion_lineas.js](#) (casos 38–51), que corre con [node run_tests.mjs](#).

0. Cómo usar este documento

Este archivo es **autocontenido**: se puede abrir en una sesión nueva sin más contexto que el repositorio. Documenta las desviaciones detectadas entre la política de riesgo vigente y lo que hace hoy el código, como paso previo a **encapsular el motor de otorgamiento en un servicio (MS/API) independiente**.

Antes de extraer el motor conviene cerrar estos puntos: cada uno cambia el **contrato** del futuro servicio (qué nivel devuelve, quién puede aprobar, qué reglas existen). Extraer primero y corregir después obliga a versionar el contrato dos veces.

Cada hallazgo trae: **qué dice la política** · **qué hace el código** · **evidencia** ([archivo:línea](#)) · **impacto** · **corrección propuesta** · **decisión pendiente**. Las decisiones pendientes eran de negocio (Datamart / Riesgo de Factoring Security), no técnicas, y **las seis se tomaron el 11-09-2026**; el recuadro al inicio de cada hallazgo dice cuál fue y qué se hizo con ella.

Convención de referencias: todas las líneas apuntan a [pipeline_comercial.jsx](#) salvo que se indique otro archivo.

1. Resumen de hallazgos

#	HALLAZGO	SEVERIDAD	EFFECTO
~~INC-01~~	~~Homologación de niveles invertida~~	RESUELTO	NV pasa a identidad: el nivel que la regla configura es el que se exige. El nivel es configuración de la regla, no algo que el motor deba transformar
~~INC-02~~	~~Sólo aprueba el nivel exacto; los superiores no~~	RESUELTO	Decidido por el negocio el 11-09-2026 e implementado: aprueba cualquier nivel igual o superior de la misma área , sin tope. Ver el cierre de INC-02
~~INC-03~~	~~El área de Operaciones no puede aprobar nada~~	RESUELTO	El ÁREA la declara la regla y el NIVEL su tramo; con ese par se buscan en la lista de usuarios los de esa área con ese nivel o superior
~~INC-04~~	~~Faltan 4 reglas del catálogo (C47–C50)~~	RESUELTO	Decidido por el negocio el 11-09-2026 e implementado: C47–C50 son del par C–D , se evalúan una vez por deudor y se visan por deudor. El catálogo corre las 79 reglas de la política
~~INC-05~~	~~Conviven dos modelos de atribución paralelos~~	RESUELTO	Decidido por el negocio el 11-09-2026: el monto de la operación sí cambia el nivel de aprobación. Ese factor entró al motor de reglas como PISO_ATRIB_MONTO (el requisito es el mayor entre el tramo y el piso) y el modelo de causas de desvío se retiró entero
~~INC-06~~	~~Nivel Comité sin representación~~	RESUELTO	Decidido por el negocio el 11-09-2026: el Comité no es un nivel aparte ni una cuenta del sistema . C05 se configura como todas, con su par (área, nivel) → Riesgo N5
~~INC-07~~	~~Artefactos del repo desactualizados~~	RESUELTO	atribuciones_otorgamiento.json regenerado desde la app corregida (79 criterios · 134 tramos, ninguno sin aprobador) y los dos .xlsx movidos a Legado/ con su README

2. Fuentes cotejadas

FUENTE	ROL	REFERENCIA
Specs_Procesos/Otorgamiento/Spec_Proceso_Calificacion_Otorgamiento_Verificacion_v1.1.pdf	Política vigente (18-08-2026, reemplaza íntegramente a v1.0)	Normativa
Integraciones/spec_s3_otorgamiento.md	Contrato de datos de entrada (Activo A16) + upsert intradía A22	Normativa
pipeline_comercial.jsx	Implementación actual (demo)	Auditada
atribuciones_otorgamiento.json	Export de atribuciones	Desactualizado (ver INC-07)
Rules_Cliente2.xlsx , Rules_Deudor.xlsx	Maestros originales	Legado (ver INC-07)

Supuesto de trabajo: la política v1.1 es normativa; el código es la parte que se ajusta. Si en algún punto la decisión fuera la inversa (el código refleja un acuerdo posterior no documentado), hay que actualizar la spec y anotarlo aquí.

3. Hallazgos

INC-01 · Homologación de niveles invertida

Qué dice la política. Spec §3: la escala es N1..N5 más Comité, donde **N5 es la máxima atribución individual**:

NIVEL	ROL	ÁREA
N1	Jefe de Grupo Comercial	Comercial
N2	Gerente Comercial	Comercial
N3	Gerente General	General
N4	Jefe de Riesgo	Riesgo
N5	Subgerente de Riesgo	Riesgo
COMITÉ	Comité de Crédito	Riesgo

La spec menciona además una homologación a un módulo interno donde 1 = máxima: `nivelModulo = 6 - N` (Comité → 1).

Qué hace el código. El constructor del catálogo aplica esa conversión al **guardar** el nivel de cada tramo:

```
// pipeline_comercial.jsx:9816
const NV = (N) => 6 - N, MM = 1e6;
```

Pero **todo lo que consume ese número lo interpreta en la convención de la política** (5 = máxima):

```
// pipeline_comercial.jsx:10584
const NIVEL_ROL = {
  1: { rol: "Jefe de Grupo Comercial", area: "comercial" },
  ...
  5: { rol: "Subgerente de Riesgo", area: "riesgo" },
};
// pipeline_comercial.jsx:912 - ATRIB_USUARIO: JG comercial:1 ... SR riesgo:5
// pipeline_comercial.jsx:966 - puedeAprobarExc: lv >= nivelReq (a mayor número, más atribución)
```

Es decir: se **escribe** en convención módulo y se **lee** en convención política. La conversión queda sin contraparte.

Evidencia. Nivel que la política asigna vs. rol al que efectivamente rutea el código:

REGLA	POLÍTICA	GUARDADO	APROBADOR EFECTIVO HOY
C01 Pagaré firmado	N1 (Operaciones)	5	Subgerente de Riesgo
C07 Cupo en línea	N2 / N4	4 / 2	Jefe de Riesgo / Gerente Comercial
C09 Nota cliente < 3,7	N4 (Riesgo)	2	Gerente Comercial
C17 Mora Equifax	N2 / N3	4 / 3	Jefe de Riesgo / Gerente General
C22 Infracciones laborales	N2 / N3 / N4 / N5	4 / 3 / 2 / 1	Jefe de Riesgo → Gerente General → Gerente Comercial → Jefe de Grupo Comercial
C23 Mora interna > 25d	N1	5	Subgerente de Riesgo
D01 Nota deudor < 3,7	N4	2	Gerente Comercial
D18 Socios comunes C-D	N5	1	Jefe de Grupo Comercial
O01 Spread bajo banda	N1	5	Subgerente de Riesgo

El caso de C22 muestra el efecto con claridad: **a mayor monto de infracciones, menor jerarquía del aprobador**. La escala de severidad quedó invertida.

Impacto. Alcanza a **67 reglas** con tramo de excepción y **130 tramos de excepción** (de 180 tramos en total) — es decir, prácticamente todo el catálogo excepcionable. Medido en runtime; contarlos con `grep` da otro número porque el catálogo se arma en un IIFE (ver §7.4). Hoy esos 130 tramos rutean a **comercial 76 · riesgo 54 · operaciones 0**, y ese cero es INC-03 medido. Consecuencias:

- Las excepciones de Riesgo se rutean a Comercial y viceversa (`NIVEL_ROL[n].area` cambia con el nivel).
- La lista de aprobadores hábiles (`aprobadoresExc`, línea 972) y las notificaciones (`solicitarAprobacionExc`, línea 4923) apuntan a personas equivocadas.
- El JSON exportado desde Mantenedores (`buildAtribucionesJSON`, línea 11469) publica esos niveles incorrectos hacia afuera.

Corrección propuesta. Eliminar la conversión: `NV` pasa a ser identidad (`const NV = (N) => N`). Verificado tramo por tramo, **con esa sola corrección las 67 reglas quedan alineadas con la spec §7–§9**, sin tocar ninguna otra cosa. Ejemplos: C07 → N2/N4 ✓, C17 → N2/N3 ✓, C22 → N2/N3/N4/N5 ✓, C23 → N1 ✓, D18 → N5 ✓, O01–O04 → N1 ✓.

La única excepción es C05 (ver INC-06).

INC-01 e INC-03 van juntos. Corregir sólo `NV` cambia el **área responsable de 107 de los 130 tramos** de excepción: `NIVEL_ROL` mapea 1–3 a comercial y 4–5 a riesgo, así que invertir el nivel cruza las áreas en todos los tramos salvo los que ya están en el nivel 3. Si el paso 7 del plan (§6) se aplica sin el paso 8, las excepciones que hoy van al aprobador equivocado siguen yendo al equivocado, sólo que al otro. **Los dos pasos son un mismo cambio.**

DECISIÓN TOMADA (11-09-2026) — INC-01 CERRADO. «A mayor gravedad se requiere mayor jerarquía, pero esto no es cuestión del código sino de la configuración definida en cada regla.» El nivel que cada criterio necesita es un dato de la regla; el motor no debe transformarlo. `NV` queda como **identidad** y lo que el catálogo declara es lo que se exige. Se conserva el nombre `NV` para no tocar las 130 llamadas del catálogo.

IMPLEMENTADO. Verificado en runtime: C01 «Existencia de Pagaré Firmado» pasa de exigir el **Subgerente de Riesgo** a exigir **Operaciones N1**; C21 «Deuda Morosa ACHEF +180» pasa del **Jefe de Grupo Comercial** al **Subgerente de Riesgo**; C22 «Infracciones Laborales» escala N2/N3/N4/N5 por monto, de menor a mayor. Casos 38–40 de `tests_asignacion_lineas.js`.

INC-02 · Sólo aprueba el nivel exacto, no los superiores

Qué dice la política. Spec §3, textual: «el nivel de cada regla es el **MÍNIMO** requerido; cualquier nivel superior puede autorizar».

Qué hace el código. Exige además que el nivel del usuario esté **explícitamente definido en los tramos de esa misma regla**:

```
// pipeline_comercial.jsx:959
function nivelesAprobArea(regla, area) {
  const s = new Set();
  ((regla && regla.tiers) || []).forEach((t) => { if (t[1] === "excepcion") { const niv = t[2]; const nr = NIVEL_ROL[niv] || NIVEL_ROL[4];
  return s;
  }
  // pipeline_comercial.jsx:966
  function puedeAprobarExc(code, regla, nivelReq) {
    if (code === "ADMIN") return true;
    const nr = NIVEL_ROL[nivelReq] || NIVEL_ROL[4];
    const lv = atribDe(code).atrib[nr.area];
    return lv != null && lv >= nivelReq && nivelesAprobArea(regla, nr.area).has(lv);
  }
}
```

El mismo criterio aparece en `aprobadoresDe` (línea 913), que compara con igualdad estricta:

```
const aprobadoresDe = (area, nivel) => Object.keys(ATRIB_USUARIO).filter((k) => USERS[k] && (k === "ADMIN" || ATRIB_USUARIO[k].atrib[area])
```

Impacto. Una regla cuyo único tramo de excepción es N4 (por ejemplo C09 o D01) sólo puede ser visada por el **Jefe de Riesgo**. El **Subgerente de Riesgo (N5)**, que la política define como máxima atribución individual, queda excluido: cumple `lv >= nivelReq` pero su nivel no figura entre los tramos de esa regla. En la

práctica desaparece la sustitución por jerarquía superior y cada excepción depende de una única persona.

Corrección propuesta. Eliminar la condición `nivelesAprobArea(...).has(lv)` y dejar `lv >= nivelReq` como única regla (más el super-admin). Eso implementa literalmente el «mínimo requerido» de la spec.

DECISIÓN TOMADA (11-09-2026) — INC-02 CERRADO. El negocio confirmó: aprueba cualquier nivel **igual o superior dentro de la misma área, sin tope**. Dos personas con el mismo rol aprueban las dos; si un cargo queda vacante —o la persona está de vacaciones— la jefatura de su área lo cubre: un Gerente Comercial (N2) visa lo que le tocaba al Jefe de Grupo (N1). La escalada **no cruza áreas**: un Gerente General no visa una excepción de Riesgo por ser superior en la jerarquía comercial, porque son dos atribuciones distintas y no una sola escalera.

IMPLEMENTADO. `nivelesAprobArea` se eliminó y `puedeAprobarExc` quedó en `lv != null && lv >= nivelReq` (más el super-admin). Junto con esto, la atribución dejó de colgar del CÓDIGO de usuario y pasó a derivarse del **ROL** (`ROL_ATTRIB` → `atribDeRol` → `atribDe`): antes GG: { `atrib: { comercial: 3 }` } funcionaba sólo porque había un usuario por rol y el código era su abreviatura, así que cambiarle el cargo a alguien no le cambiaba la atribución. Los casos 35–37 de `tests_asignacion_lineas.js` fijan las tres decisiones.

Queda **fuera de este cierre** el ruteo: con INC-01 sin resolver, el nivel que un tramo pide sigue saliendo invertido, así que la escalada funciona correctamente sobre un nivel que todavía puede ser el equivocado.

Nota: el propio `atribuciones_otorgamiento.json` documenta la convención del código, no la de la spec: «el nivel del tramo aplicable es el aprobador responsable; un SUPERIOR sólo puede aprobar (sustitución de emergencia) si su nivel está definido en el risk tier de la misma regla». Al cerrar este punto hay que alinear también ese texto.

INC-03 · El área de Operaciones no puede aprobar nada

Qué dice la política. Spec §7 clasifica reglas como **EXC-OPS** (excepcionables por el área de Operaciones): C01–C04 —pagarés e información financiera— son de esa área.

Qué hace el código. El catálogo respeta el área en la regla (`R(101, "C01", "operaciones", ...)`, línea 9825 y siguientes), pero el ruteo de aprobación no usa ese campo: usa `NIVEL_ROL[nivel].area`, y `NIVEL_ROL` sólo contiene **comercial y riesgo** (línea 10584). Como `puedeAprobarExc` resuelve el área por nivel:

```
const nr = NIVEL_ROL[nivelReq] || NIVEL_ROL[4];
const lv = atribDe(code).atrib(nr.area); // nr.area ∈ { comercial, riesgo }
```

el usuario aprobador de Operaciones — `OP: { tipo: "aprobador", atrib: { operaciones: 5 } }` (línea 878)— nunca obtiene un `lv` distinto de `undefined` y jamás resulta hábil para visar una excepción.

El código incluso reconoce la divergencia y la muestra en pantalla en vez de resolverla:

```
// pipeline_comercial.jsx:10779
const niv = x.nivel || 4; const nr = NIVEL_ROL[niv] || NIVEL_ROL[4]; const otraArea = nr.area !== x.regla.area;
```

Impacto. El área de Operaciones existe en el catálogo, en los mantenedores y en la matriz de gravedad, pero está muerta en el flujo de aprobación. Las excepciones de C01–C04 terminan en Comercial o Riesgo según el nivel que les tocó por INC-01.

Corrección propuesta. Separar los dos conceptos, que hoy están fusionados en `NIVEL_ROL`:

- **Área responsable** → siempre `regla.area` (`riesgo` | `comercial` | `operaciones`).
- **Nivel requerido** → el del tramo aplicable.
- **Rol aprobador** → función de `(área, nivel)`, no de `nivel` solo.

Es decir, reemplazar `NIVEL_ROL: nivel → {rol, área}` por una matriz `ROL_POR_AREA_NIVEL: (área, nivel) → rol`, y hacer que `puedeAprobarExc` compare contra `regla.area`.

DECISIÓN TOMADA (11-09-2026) — INC-03 CERRADO. «La regla define el área y el nivel; eso marca las definiciones y los requerimientos. El sistema debe ir a la lista de usuarios, buscar al usuario del área que se requiere y del nivel que se requiere, y ahí se obtienen los candidatos.» El nivel es una forma de segmentar la atribución en una escala, y todas las áreas usan la misma.

IMPLEMENTADO. `puedeAprobarExc` lee `regla.area` en vez de `NIVEL_ROL[nivelReq].area`. Una regla **sin área declarada no la aprueba nadie**: el default silencioso a «riesgo» que tenía la primera versión de este arreglo escondía una regla mal configurada detrás de una aprobación que igual ocurría. `rolDeAreaNivel(area, nivel)` le pone nombre al cargo requerido, y si no hay uno exacto nombra al primero del área que lo alcance — la misma escalada que aplica el gate.

Medido tras el cambio, ningún par (área, nivel) del catálogo queda sin aprobador posible:

ÁREA · NIVEL	TRAMOS	QUIÉN PUEDE
comercial · N1	18	Sofía Herrera, Dante Montes, Federico Diaz
comercial · N2	11	Dante Montes, Federico Diaz
operaciones · N1	3	Andrés Mella
operaciones · N2	1	Andrés Mella
riesgo · N1–N4	78	Carolina Vergara, Paula Reyes
riesgo · N5	19	Paula Reyes

Observación de configuración, no de código: el área Riesgo sólo tiene cargos en los niveles 4 y 5, así que sus 78 tramos de N1 a N4 caen todos en el Jefe de Riesgo por escalada. Si se quiere distinguirlos, es cosa de dar de alta usuarios de Riesgo en N1–N3 desde el mantenedor — exactamente el camino que el negocio definió para cubrir un par vacío.

~~INC-04~~ · Faltan 4 reglas del catálogo (C47–C50) — RESUELTO 11-09-2026

Decisión de negocio (11-09-2026): son de tipo D. C47–C50 se evalúan **una vez por cada deudor** de la operación y su visado es por deudor (`stKey = n@rut`), aunque la spec las numere en el bloque de cliente. **Implementado:** cuatro variables nuevas del par en `deudorBlock` (`cdCarteraReclamada`, `cdCarteraNC`, `cdCarteraMorosa`, `cdCxcPend`), cuatro reglas `R(147..150, ..., { porDeudor: true })` con área **comercial** y nivel **N1**, re-evaluables, y `esReglaDeudor` pasa a respetar la bandera en vez de deducir el tipo del prefijo del código —deducirlo obligaba a reenumerarlas D24–D27 y perder la trazabilidad con la spec—. **A16 actualizado:** el layout suma `CARTERA_RECLAMADA_CD`, `CARTERA_NC_CD`, `CARTERA_MOROSA_CD` y `CXC_PENDIENTES_CD` en la fila `DEUDOR` (una por par), con su diccionario y su ejemplo. **Verificado:** el catálogo corre las **79 reglas** de la política, ningún código sin implementar. Casos 46–48.

Qué dice la política. Spec §1: «la política de riesgo evalúa 79 reglas» = C01–C52 (52) + D01–D23 (23) + O01–O04 (4). Spec §7 detalla **C47–C50** — «Cartera del par C-D: reclamados / NC / mora / CxC», carácter EXC-COM, nivel N1c, re-evaluables.

Qué hace el código. El catálogo de runtime tiene **75 reglas**: 48 de cliente, 23 de deudor, 4 de operación. Faltan exactamente **C47, C48, C49 y C50**.

Verificable con:

```
awk '/^      R/{/{print}' pipeline_comercial.jsx | grep -oE "[CD0][0-9]{2}" | tr -d '"' | sort > /tmp/impl.txt
for i in $(seq -w 1 52); do grep -qx "C$i" /tmp/impl.txt || printf "C%s " $i; done; echo
# → C47 C48 C49 C50
```

Impacto. Son las cuatro reglas equivalentes a C40–C43 pero aplicadas al **par cliente-deudor** en lugar del cliente completo. Su ausencia deja sin cubrir la gestión de cartera a nivel de par, que es justamente donde se detecta el deterioro localizado en un deudor. C40–C43 (cliente) sí están implementadas (líneas 9864–9867).

Corrección propuesta. Implementarlas siguiendo el patrón de C40–C43 pero como reglas del deudor —es decir, evaluadas una vez por cada deudor de la operación, con `stKey = n@rut` — dado que son del par C-D. Requieren cuatro variables nuevas en el payload (cartera reclamada / NC / morosa / CxC del par), que **hoy no viajan** en `s3_otorgamiento.csv`.

Decisión pendiente. Confirmar si C47–C50 se evalúan por par C-D (y entonces son reglas tipo D, con visado por deudor) o si son agregadas del cliente (y entonces son tipo C). La spec §7 las lista en el bloque CLIENTE pero las nombra «del par C-D», lo que es ambiguo. De la respuesta depende el layout del archivo A16 y la forma del `stKey`.

~~INC-05~~ · Conviven dos modelos de atribución paralelos — RESUELTO 11-09-2026

Decisión de negocio (11-09-2026): «El monto sí cambia el nivel de aprobación.» O sea, el factor que el modelo A aportaba es real y tenía que conservarse; lo que sobraba era el modelo paralelo. **Implementado:** `PISO_ATRIB_MONTO` dentro del motor de reglas. Son **dos factores** y el requisito es el **mayor**: el tramo del risk tier mide cuánto se desvía la variable de riesgo, el piso mide cuánto se arriesga si ese desvío resulta cierto. `max` y no suma —sumando, dos factores medianos exigirían más que un factor extremo—, y **piso** y no reemplazo: el nivel del tramo nunca baja por el monto. Se aplica en `evaluarOtorgItems`, el único sitio que arma los items, así que todo lo que decide después lo hereda. **La tabla vieja no se pudo reutilizar.** Sus celdas eran niveles de una escalera ÚNICA en la que N4 y N5 ya eran de Riesgo: «comercial: 2» significaba *Gerente Comercial*, no *Comercial nivel 2*. Desinvertirla con `6 - N` daba `comercial: 4`, un nivel que Comercial **no alcanza** —su tope en la política es N3, Gerente General—, y toda excepción comercial sobre MM\$120 habría quedado «Sin aprobador definido». Cada columna satura en el tope de su área. El caso 40 de la suite pasó a probar cada tramo contra **cada tramo de monto**, que es lo que detecta ese error. **Retirado:** `MATRIZ_OTORG`, `TIPOS_DESVIO`, `tipoActivo`, `causasDeDeal`, `nivelReqCausa`, `puedeAccionarCausa`, `autorizarCausa`, `otorgarOperacion` y el cómputo muerto de `OtorgamientosView`. **Conservados:** `CFG_TRAMOS` y `gravedadPorMonto` (alimentan el piso) y `requiereOtorgamiento` (rutea la etapa y gatea el tab del detalle). **Un efecto lateral que la auditoría no había visto:** la cadena de acción del modelo A no sólo estaba huérfana en la vista, estaba **muerta** — `onAutorizarCausa` y `onOtorgarOperacion` se pasaban como props a `DealDrawer` y a `OtorgamientosView` y **ninguno de los dos las invocaba**. Como el motor de fondo liberaba el giro exigiendo «todas las causas autorizadas», una operación derivada a otorgamiento **manual** no tenía salida salvo que alguien la moviera a mano desde el selector del detalle. Ahora la libera `otorgamientoCompleto`, que decide con el VISADO. El texto que sigue es el diagnóstico original.

En el código hay **dos mecanismos completos y distintos** para decidir quién autoriza una operación:

	A · CAUSAS DE DESVÍO	B · MOTOR DE REGLAS
Qué evalúa	<code>TIPOS_DESVIO</code> : FL (fuera de línea), OD (otros deudores), ON, DI	Catálogo C01–C52 / D01–D23 / O01–O04
Origen del nivel	<code>MATRIZ_OTORG[gravedad][área]</code> , con gravedad por tramo de monto (<code>CFG_TRAMOS</code>)	Tramo del risk tier de cada regla
Convención	1 = máxima (crítico → riesgo:1)	5 = máxima (<code>NIVEL_ROL</code>)
Permiso	<code>puedeAccionarCausa</code> : <code>lv === nivelReq</code> (igualdad)	<code>puedeAprobarExc</code> : <code>lv >= nivelReq</code> + presente en los tramos
Disparador	<code>requiereOtorgamiento(deal)</code> — fuera de línea y/o deudores «Otro»	Evaluación de las 75 reglas sobre las variables A16
Ubicación	líneas 897–972	líneas 9791–10107

Impacto. Tres consecuencias concretas:

1. **El modelo A está huérfano en la UI.** `OtorgamientosView` (línea 11285) calcula `items` , `enOtorg` y `esPipeline` a partir de las causas... y **no los usa**: el JSX sólo renderiza `<VisadoClienteView>` , que es el modelo B. Son ~10 líneas de cómputo muerto en cada render.
2. **La matriz del modelo A es inalcanzable.** `MATRIZ_OTORG` (línea 854) exige `riesgo:1` para gravedad crítica, pero ningún usuario tiene ese nivel (`RG` es riesgo:4, `SR` es riesgo:5) y `puedeAccionarCausa` compara por `igualdad`. Resultado: las causas críticas sólo las puede accionar `ADMIN` , y las **leves** (que piden riesgo:5) las acciona el **Subgerente de Riesgo** —la máxima atribución para el caso más benigno.
3. `requiereOtorgamiento` **sigue siendo relevante** (líneas 1732–1739) para decidir el ruteo a la etapa Otorgamiento y para `causasDeDeal` , pero su resultado ya no gobierna la aprobación.

Corrección propuesta. Elegir un modelo único para el servicio. Recomendación: **conservar B** (es el que implementa la política v1.1 y el que la UI usa) y **reexpresar FL y OD como reglas del catálogo** —de hecho ya existen equivalentes: C07 cubre el cupo de línea y la clasificación de deudor pertenece al inbound. Con eso, `MATRIZ_OTORG` , `CFG_TRAMOS` , `TIPOS_DESVIO` , `causasDeDeal` , `gravedadPorMonto` , `nivelReqCausa` y `puedeAccionarCausa` se retiran.

Decisión pendiente. Confirmar que la gravedad por tramo de monto (≤ 20 / ≤ 60 / ≤ 120 / > 120 MM) **no** debe influir en el nivel requerido. Hoy el modelo B ignora el monto de la operación: el nivel depende sólo del valor de la variable de riesgo. Si el monto sí debe escalar la atribución, hay que incorporarlo al modelo B como un modificador explícito y documentarlo en la spec, en vez de mantenerlo en un modelo paralelo.

~~INC-06~~ · El nivel Comité no tiene representación — RESUELTO 11-09-2026

Decisión de negocio (11-09-2026): el Comité no es un nivel aparte. «El comité de riesgo no es un usuario; ahí hay que definir un usuario y un nivel como en todas las reglas. Todas son iguales.» O sea: no se agrega un nivel 6 ni un token `"COMITE"` , y el motor **no** emite un resultado de otra naturaleza. **Implementado:** C05 deja de ser la única regla con el nivel escrito a mano (`[["", "excepcion", 1]] → NV(5)`), quedando en **Riesgo N5**, la máxima atribución individual — que es lo que corresponde a la decisión más estructural del proceso y lo que exige el principio de INC-01 (a mayor gravedad, mayor jerarquía). **Verificado:** ninguna regla del catálogo declara un nivel fuera de N1..N5. Caso 49.

Qué dice la política. Spec §3 define **COMITÉ** como un nivel por encima de N5, área Riesgo, para «*constitución de líneas nuevas y cambios estructurales*». Spec §7 asigna a **C05 (Línea Cliente Nuevo)**: «*Sin línea → constitución vía COMITÉ*».

Qué hace el código. `NIVEL_ROL` (línea 10584) sólo define 1..5; no hay entrada para Comité. C05 es además la **única regla que fija su nivel de forma literal**, sin pasar por `NV()` :

```
// pipeline_comercial.jsx:9829
R(105, "C05", "riesgo", "ClientSegmentation", "Línea Cliente Nuevo",
  "Cliente nuevo sin línea de crédito aprobada – requiere constitución de línea (Comité de Crédito)",
  [[(v) => v.clienteNuevo, "excepcion", 1]]),
```

Ese `1` corresponde a la homologación «Comité → 1» de la spec, pero `NIVEL_ROL[1]` es **Jefe de Grupo Comercial**, el rol de menor atribución del sistema.

Impacto. La constitución de una línea nueva —la decisión más estructural del proceso— queda a criterio del aprobador de menor jerarquía. Además, al corregir INC-01 (`NV` identidad), C05 seguirá roto porque su nivel está escrito a mano: es el único caso que no se arregla solo.

Corrección propuesta. Añadir el nivel Comité como valor propio por encima de N5 (`6` , o mejor un token `"COMITE"` para no depender del orden numérico), con su rol y área, y apuntar C05 a él. Si se usa un token, `puedeAprobarExc` necesita un orden explícito en vez de comparación numérica.

Decisión pendiente. Definir si el Comité es un **usuario** del sistema (una cuenta que visa en la bandeja) o un **estado externo** (la operación sale del flujo, va al módulo de Solicitud de Línea al Comité —ver `Specs-Procesos/Líneas/Análisis_Solicitud_Línea_Comite.md` — y vuelve con una resolución). La segunda opción es la que sugiere el resto del diseño y cambia el contrato del servicio: el motor no devolvería «excepción nivel Comité» sino «requiere constitución de línea», un resultado de otra naturaleza.

~~INC-07~~ · Artefactos del repositorio desactualizados — RESUELTO 11-09-2026

Hecho. `atribuciones_otorgamiento.json` regenerado desde la app ya corregida: **79 criterios · 134 tramos de excepción · ninguno sin aprobador**, con la `descripcion` que describe la convención vigente (la regla declara el área, el tramo el nivel, aprueba ese nivel o superior de la misma área, la escalada no cruza áreas). Para que no vuelva a desfasarse dos meses, la regeneración dejó de ser un gesto manual: `node build_app.mjs && PLAYWRIGHT_BROWSERS_PATH=/opt/pw-browsers node regenerar_atribuciones.mjs`. Los dos `.xlsx` se **conservaron por trazabilidad** en `Legado/` con el sufijo `_v0` y un README que dice por qué no son fuente. El texto que sigue es el diagnóstico original.

`atribuciones_otorgamiento.json` (raíz del repo)

- Declara `"generado": "2026-07-10"` y `"total_criterios": 59`.
- Contiene el **catálogo anterior** (reglas 1–59, con nombres tipo «*Clasificación Matriz Cliente — Categoría 1*», «*Protestos Vigentes Banco BICE*», «*Morosidad BICE Factoring*»), no el Modelo de Riesgo v1.0 C/D/O.
- El runtime lo reemplaza completo en el IIFE de la línea 9901 (`REGLAS_CLIENTE.length = 0; V2.forEach(...)`), de modo que el archivo del repo **no describe lo que la aplicación ejecuta**.
- Su campo `descripcion` documenta además la convención de nivel exacto que contradice la spec (ver INC-02).
- Se regenera desde la app: Configuración → Otorgamiento → Atribuciones de aprobación → **Descargar JSON** (`buildAtribucionesJSON` , línea 11469). Conviene regenerarlo **después** de cerrar INC-01 a INC-03, no antes.

`Rules Cliente2.xlsx` y `Rules Deudor.xlsx` (raíz del repo)

- Son los maestros originales de los que salió el catálogo anterior.
- `Rules Deudor.xlsx` tiene **10 reglas** (vs. las 23 de D01–D23) y evalúa `$DeudorScore` en escala **0–99** con umbrales 80/60 — una escala que ya fue reemplazada por la **Nota Deudor 1–5** con umbral 3,7 (invariante de dominio n° 2 en `CLAUDE.md`).
- `Rules Cliente2.xlsx` tiene 59 filas y columnas de trabajo (`Rutle Task` , `Reevaluation`) que no existen en el modelo actual.

Corrección propuesta. Marcarlos explícitamente como **legado** —moverlos a un subdirectorio `Legado/` o añadirles el sufijo `_v0` — para que nadie los tome como fuente de verdad al implementar el servicio. La fuente normativa es la spec v1.1 más `Integraciones/spec_s3_otorgamiento.md`.

Decisión pendiente. Ninguna de negocio; es higiene del repositorio. Confirmar sólo si se conservan por trazabilidad o se eliminan.

4. Verificado y consistente (no re-auditar)

Estos puntos se cotejaron y **coinciden** entre política e implementación:

PUNTO	VERIFICACIÓN
Evaluación por deudor	<code>evaluarOtorgItems</code> (línea 9791) evalúa C y O una vez, y las D una vez por cada deudor con <code>deudorBlock</code> superpuesto. Coincide con spec §2
Clave de visado <code>stKey</code>	<code>String(n)</code> para cliente/operación, <code>n + "@" + rut</code> para deudor (línea 4726). Coincide con spec §6
Reglas no re-evaluables	El set es <code>{110-122, 130-132, 202-213}</code> = C10–C22, C30–C32, D02–D13 (línea 9688). Coincide exactamente con spec §5
KNOCKOUT	C30/C31/C32 usan <code>tHard</code> → disposición <code>rechazado</code> + no re-evaluables → <code>rechFirme</code> → pérdida terminal automática (<code>useEffect</code> línea 20386). Coincide con spec §4
Rechazo re-evaluable ≠ pérdida	<code>visadoDealCalc</code> (línea 10107) separa <code>rechFirme</code> de <code>rechReev</code> ; sólo el primero produce pérdida. Coincide con spec §5
Estado agregado	<code>aprobada</code> / <code>sujeta</code> / <code>rechazada</code> según excepciones pendientes y rechazos. Coincide con spec §6
Re-evaluación v1 → v2	<code>apiVarsCliente(deal, rev)</code> con <code>rev ≥ 1</code> regulariza sólo documentación, vigencias y garantías; no toca datos de buró (línea 9646). Coincide con spec §5
Las excepciones visadas no se re-abren	<code>reevaluarCliente</code> (línea 4948) no toca <code>VISADO_STATE</code> . Coincide con spec §6
Umbrales de los tramos	Los valores (MM\$5/10, MM\$25/50, 3,7, 50%, 30/60%, 8/30%, 3/20%, 3/8 factorings, 35%) coinciden con spec §7–§9. El error está en el nivel asignado, no en el umbral

5. Parámetros abiertos heredados de la spec (§13)

No son inconsistencias: son definiciones que la propia política declara pendientes. Bloquean el cierre del contrato del servicio.

REF	PARÁMETRO
C07	Tramos de excedente de línea (hoy 10%)
C08	Umbrales de variación de venta (−20% / −40%)
C13 / D05	Castigada CMF directa: ¿escala por monto o bloqueo firme?
C27	Deuda vigente TGR: N2 en vez de rechazo (confirmado en v1.1)
C37	Umbrales ratio cesión/venta (50% / 80%)
C39	Cesión a factorings pequeños: ¿excepcionable o bloqueo firme?
O01	Valores de banda de atribución y descuento máximo
O03	Semántica del 30% de CxC: ¿mínimo exigido o tope de aplicación?
—	Escala de la nota de comportamiento: 1–5 vs. 1–7 (el umbral 3,7 se mantiene)

6. Plan de trabajo sugerido

Fase 1 — Decisiones de negocio (bloqueante; no hay código que escribir antes)

1. `~INC-01`: confirmar convención única de niveles. `~ HECHO` (11-09-2026: el nivel es configuración de la regla; `NV` queda identidad).
2. `~INC-02`: confirmar que cualquier nivel superior autoriza; definir si hay tope. `~ HECHO` (11-09-2026: superior de la MISMA área, sin tope).
3. `~INC-03`: definir la escala de niveles del área Operaciones. `~ HECHO` (11-09-2026: la misma escala para todas las áreas; la regla declara área y nivel).
4. `~INC-04`: definir si C47–C50 son reglas de cliente o de par C-D, y las variables que necesitan en A16. `~ HECHO` (11-09-2026: son tipo D — por par, visado por deudor; A16 suma las cuatro columnas `*_CD`).
5. `~INC-05`: confirmar si el monto de la operación debe escalar la atribución. `~ HECHO` (11-09-2026: sí — entra como piso por área en el motor de reglas y el modelo paralelo se retira).
6. `~INC-06`: definir si Comité es usuario del sistema o salida a otro proceso. `~ HECHO` (11-09-2026: ninguna de las dos — el Comité no es un nivel aparte; C05 se configura como todas, Riesgo N5).

Fase 2 — Corrección en el módulo actual (cada punto es una edición acotada)

1. `~ NV` → identidad `~ HECHO`, C05 incluida: dejó de ser la única regla con el nivel escrito a mano.
2. `~` Reemplazar `NIVEL_ROL` por `ROL_POR_AREA_NIVEL`. `~ HECHO`: `ROL_ATTRIB` (rol → área + nivel) es la escalera, `rolDeAreaNivel` su inverso para nombrar el cargo, y `puedeAprobarExc` compara contra `regla.area`.
3. `~` Quitar la restricción `nivelesAprobArea(...).has(lv)`. `~ HECHO`: la función se eliminó y `puedeAprobarExc` quedó en `lv >= nivelReq`.
4. `~` Implementar C47–C50. `~ HECHO`: reglas del par (`porDeudor`), variables `cd*` en `deudorBlock` y columnas `*_CD` en A16.
5. `~` Retirar el modelo de causas de desvío y su código muerto en `OtorgamientosView`. `~ HECHO`.
6. `~` Regenerar `atribuciones_otorgamiento.json` y marcar los `.xlsx` como legado. `~ HECHO`: regeneración automatizada (`regenerar_atribuciones.mjs`) y `.xlsx` en `Legado/`.

Fase 3 — Encapsulamiento

1. Recién entonces extraer el motor: catálogo + evaluador + niveles quedan estables y el contrato del servicio se define una sola vez.

Verificación tras cada cambio (según `CLAUDE.md`): `tsc --jsx preserve --allowJs --noEmit --skipLibCheck` sin errores TS1, y el chequeo de duplicados de nivel módulo vacío. (Los comandos exactos, y los pasos 3 a 7 que este documento aún no conocía, están en la sección *Verificación* de `CLAUDE.md`: es el único sitio donde se escriben. El patrón del paso 2 cambió el 18-09-2026 y no se copia acá para que no vuelva a quedar viejo.)

7. Cómo reproducir la verificación

Las líneas citadas en este documento están ancladas al commit `212046f`. `pipeline_comercial.jsx` es un archivo único de ~21.000 líneas que crece con cada cambio, así que los números se desplazan. Los comandos de abajo **buscan por símbolo** en vez de citar una línea: siguen funcionando aunque el archivo se mueva, y sirven para re-ancorar el texto. Si vas a actualizar este documento, saca los números de acá y no del texto anterior.

```
# 1) Reglas implementadas en runtime (esperado HOY: 79 = 52 C + 23 D + 4 O. Antes de INC-04: 75)
awk '/^      R\(\{print\}' pipeline_comercial.jsx | grep -oE "[CDO][0-9]{2}" | tr -d '"' | sort > /tmp/impl.txt
wc -l /tmp/impl.txt

# 2) Reglas de cliente ausentes respecto de C01-C52 (esperado HOY: ninguna. Antes: C47 C48 C49 C50)
for i in $(seq -w 1 52); do grep -qx "C$i" /tmp/impl.txt || printf "C%s " $i; done; echo

# 3) Puntos clave del ruteo de niveles – por simbolo, no por linea
grep -n 'const NV = (N) => N' pipeline_comercial.jsx # INC-01 · identidad (antes: 6 - N)
grep -n -A6 '^function puedeAprobarExc' pipeline_comercial.jsx # INC-02 · el gate: lv >= nivelReq, misma area
grep -n '^function nivelesAprobArea' pipeline_comercial.jsx # INC-02 · debe salir VACIO (se elimino)
grep -n -A8 '^function rolDeAreaNivel' pipeline_comercial.jsx # INC-03 · el inverso (area, nivel) -> cargo
grep -n 'R(105, "C05"' pipeline_comercial.jsx # INC-06 · ya rutea por NV(5), no por un 1 literal
grep -n 'porDeudor' pipeline_comercial.jsx # INC-04 · las cuatro reglas del par
grep -n 'MATRIZ_OTORG\|TIPOS_DESVIO\|puedeAccionarCausa' pipeline_comercial.jsx # INC-05 · debe salir VACIO (se retiro)
grep -n -A6 '^let PISO_ATTRIB_MONTO' pipeline_comercial.jsx # INC-05 · el piso por monto que lo reemplaza

# 4) Conteos que este documento afirma, medidos en RUNTIME (el catalogo se arma en un IIFE:
# contarlos con grep da otro numero). Requiere el HTML construido – ver CLAUDE.md.
# Esperado HOY (cerrados INC-01/02/03/04/06; retiradas C47-C50 el 14-09; + 005 y 006 propias):
# 77 reglas · 183 tramos · 133 tramos de excepcion · 69 reglas con excepcion, y NINGUN tramo sin
# aprobador. De esas 77, 75 son de la politica v1.0 y DOS son del proceso -005 y 006, las dos de
# Operaciones (ver mas abajo)–. El reparto por area ya no se lee de NIVEL_ROL –que dejo de
# decidir– sino del area que declara cada regla: riesgo 97 · comercial 29 · operaciones 7.
# Valores previos, para comparar: 75 reglas · 180 tramos · 130 de excepcion · 67 reglas con
# excepcion, ruteados a comercial 76 / riesgo 54 / operaciones 0 – ese 0 era INC-03 medido.
# Pegar en la consola del navegador con pipeline_comercial.html abierto:
# const T = REGLAS_CLIENTE.flatMap(r => r.tiers || []);
# const E = REGLAS_CLIENTE.flatMap(r => (r.tiers||[]).filter(t => t[1]===“excepcion”).map(t => ({ r, t })));
# console.log({ reglas: REGLAS_CLIENTE.length, tramos: T.length, tramosExc: E.length,
# reglasConExc: REGLAS_CLIENTE.filter(r => (r.tiers||[]).some(t => t[1]===“excepcion”)).length,
# // el area sale de la REGLA, no del nivel: NIVEL_ROL dejo de decidir con INC-03
# area: E.reduce((a,{r,t}) => (a[r.area] = (a[r.area]||0)+1, a), {}),
# huerfanos: E.filter(({r,t}) => !aprobadoresExc(r, t[2])).length).map(({r,t}) => r.cond+“ N”+t[2]) });
# Medido el 11-09-2026: { reglas: 79, tramos: 184, tramosExc: 134, reglasConExc: 71,
# area: { riesgo: 97, comercial: 33, operaciones: 4 }, huerfanos: [] }

# 5) Texto de la politica vigente (requiere pypdf)
python3 -c "from pypdf import PdfReader; print('\n'.join((p.extract_text() or '') for p in PdfReader('Specs_Procesos/Otorgamiento/Spec_Proc
```

8. Criterios propios del proceso (no vienen de la política v1.0)

La política v1.0 define 79 reglas —75 vigentes, tras retirarse C47–C50 por quedar dominadas por C40–C43— y el catálogo las implementa todas. **Dos criterios no salen de ese documento:** los agrega este proceso, y por eso se cuentan aparte —el caso 46 mide la cobertura de la política y dejaría de medirla si sumara reglas propias al mismo total—. Los dos son del área de **Operaciones**, y no por casualidad: la política v1.0 la escribió Riesgo, y lo que le falta es justamente el control documental del acto de cursar.

005 · Contrato firmado por cliente de la operación — Operaciones N3 (13-09-2026)

Por qué existe. Ninguna operación se cursa sin constancia de que el cliente autorizó la cesión. Esa constancia existía en el flujo —el cliente firma en el portal— pero no era un criterio: no se veía en la mesa de otorgamiento, no tenía aprobador y no dejaba nada que auditar cuando la oferta se cerraba en papel.

Cómo se satisface. El criterio existe **siempre**, en las dos vías de publicación, y lo que cambia es la evidencia:

VÍA DE PUBLICACIÓN	EVIDENCIA	QUIÉN LA CREA
Electrónica (correo)	La autorización del cliente en el portal, con su actor y su hora	El cliente, al firmar. El criterio queda aprobado sin que nadie lo vise
Física (contrato en papel)	El comprobante del contrato firmado, adjunto como respaldo	El ejecutivo lo adjunta y Operaciones N3 lo visa

La evidencia electrónica se lee por [aprobacionFormalCliente](#) y no por una bandera suelta: es el mismo gate que ya sabe que **reabrir revoca la firma**, así que si el paquete cambia, lo firmado deja de describir lo que se va a cursar y la evidencia cae con él. En la vía física la firma del cliente **no** satisface el criterio: el papel se firmó fuera del sistema y no hay nada que este pueda dar por cierto.

Por qué Operaciones N3. Es viabilidad operativa del curse —la familia de los pagarés C01–C03— y no un trámite de mesa: sin esa constancia la cesión no es oponible al deudor.

Cargo nuevo. Operaciones tenía un solo cargo, en N5, así que un requisito N3 se cubría por escalada y no se distinguía del que sí necesita la máxima atribución del área. Se dio de alta **Jefe de Operaciones (N3)** —la misma observación que este documento dejó anotada para Riesgo, que sólo tiene cargos en N4 y N5—.

Dar de alta el cargo no basta: [atribDe](#) devuelve atribución vacía a quien no figure en el padrón de personas, así que el primer intento creó el cargo y la excepción seguía cayendo en el N5. El caso 85 fija las dos mitades: que el cargo exista y que alguien lo ocupe.

Variable. `contratoEvidencia` no viene de la API de riesgo sino de la propia operación, así que **no se congela con la versión**: la versión es la foto de lo que dijo el origen externo, y esto es un hecho que este sistema conoce y que cambia dentro de la misma revisión. Publicar en papel abre O05 en el acto; leído del snapshot, el criterio no aparecería hasta la próxima reevaluación —o sea, después de girar—. Se superpone **una sola** variable, a propósito: la puerta se abre para lo que la operación posee, no para reescribir el padrón de riesgo desde la UI.

Cache. `publicacion`, `clienteAcepto` y `reabierta` entraron a la clave de `visadoKey`: publicar una oferta que ya estaba en «oferta» no cambia la etapa, así que sin eso el visado cacheado seguía siendo el de antes de publicar.

O06 · Monto Cedido Igual al Monto del Documento — Operaciones N5 / N3 (14-09-2026)

Por qué existe. El usuario lo planteó como un recuerdo: «me parece que hay una regla en el motor de otorgamiento que revisa que el monto de cesión sea = monto de la factura, en el área de operaciones». **No existía.** Buscado en el catálogo vigente y en el legado (`Legado/Rules Cliente2_v0.xlsx` , la fuente de las 75 reglas): la política trae **dos** criterios sobre cesiones y los dos son de **concentración**, no de monto por documento —

CÓDIGO	QUÉ MIDE	ÁREA
C37 · Existencia Ratio Venta vs. Monto Cedido Alto LM	qué proporción de su venta cede el cliente a factoring	comercial
C39 · Alta Cesión a Factorings Pequeños	qué proporción le cede a factorings privados chicos	comercial

— y el área de Operaciones sólo tenía O05. El control faltaba **justo donde el dato podía romperse**.

Por qué no se veía. Hasta el 14-09-2026 las 1.300 cesiones del A2 eran por el **total exacto** del documento, así que la comparación se cumplía siempre. Un control que nunca puede fallar no se echa de menos: el hueco de la regla y el del dato se tapaban mutuamente. Al reconciliar el A2 contra el A1 y admitir cesiones parciales, los dos quedaron a la vista.

Cómo funciona. `varsOperacionCli` mide, documento a documento, las facturas de la oferta contra el registro de cesiones, y devuelve dos variables. Los tramos van a niveles distintos porque no son el mismo hecho:

TRAMO	QUÉ PASÓ	NIVEL
<code>cesionExcedida > 0</code>	una cesión por más que la factura: no es una diferencia, es un crédito que no existe	Operaciones N5
<code>cesionParcial > 0</code>	una cesión parcial : el crédito quedó con dos dueños y cobrarlo sería compartir la cobranza	Operaciones N3

El primer tramo el activo lo valida en origen y no debería llegar nunca. Se controla igual: **un control que sólo cubre lo que ya sabemos que pasa no controla nada**.

Relación con O05. Son el par documental del acto de ceder: O05 comprueba que **exista** la autorización del contrato, O06 que lo cedido **coincida** con lo que se va a comprar. Caso 96.